Notes:

- Cheating or Trying to Cheat may disqualify you from the course.
- ChatGPT or any AI tool is not allowed in this exam. Use only HI (Human Intelligence)!

Question 1) (10 **Points)** The following sequential C code processes a list of N web URLs. For each URL, it downloads the content and then counts the occurrences of a specific keyword.

```
struct PageData {
    char* content;
    int keyword_count;
};

// Assume this function exists
char* download_content(const char* url);
// Assume this function exists
int count_keywords(const char* text, const char* keyword);

void process_urls_sequentially(const char* urls[], int N, const char* keyword) {
    long total_keywords = 0;
    for (int i = 0; i < N; i++) {
        char* content = download_content(urls[i]);
        if (content) {
            total_keywords += count_keywords(content, keyword);
        }
    }
    printf("Total keywords found: %ld\n", total_keywords);
}
```

The download_content() function can be slow due to network latency. Rewrite a `process_urls_sequentially` function to parallelize the execution using multiple threads. Each thread should process a subset of the URLs. Ensure that the total keyword count is aggregated correctly.

Question 2) (10 **Points)** A system has three threads (T1, T2, T3) and three resources (R1, R2, R3), each protected by a separate mutex lock (L1, L2, L3 respectively). The threads attempt to acquire locks in the following order to perform their tasks:

- T1: Needs R1 then R2. Acquires L1, then L2.
- T2: Needs R2 then R3. Acquires L2, then L3.
- T3: Needs R3 then R1. Acquires L3, then L1.

Identify the critical sections for each thread, Can a deadlock occur in this system? If yes, describe a specific sequence of lock acquisitions by T1, T2, and T3 that leads to a deadlock.

Question 3) **(10 Points)** Answer the following questions:

a) What are the main criteria for evaluating locks in concurrent systems? Briefly explain each.

b) What is a race condition, and what is mutual exclusion? Explain each concept briefly.

Question 4) **(10 Points)** Answer the following MCQs and **explain why you choose this choice. Assume no syntax errors for any code snippet**

1. Two threads run through this code: What is the value of i ?	int i = 0; // global variable
	if (i == 0)
	  i++;

a) 0	
b) 1	
c) 2	
d) 1 or 2	
e) None of the above	

2. Two threads run through this code: What is the value of i?	mutex_t m;
	int i = 0;
	if (i == 0) {
	  pthread_mutex_lock(&m);
	  i++;
	  pthread_mutex_un;ocl(&m);
	}

a) 0	
b) 1	
c) 2	
d) 1 or 2	
e) None of the above	

3. Choose all possible outputs, assuming no syntax errors	volatile int counter = 1000;
	void *worker(void *arg) {
	  counter--;
	  return NULL;
	}
	int main(int argc, char *argv[]) {
	  pthread_t p1, p2;
	  pthread_create(&p1, NULL, worker, NULL);
	  pthread_create(&p2, NULL, worker, NULL);
	  pthread_join(p1, NULL);
	  pthread_join(p2, NULL);
	  printf("%d\n", counter);
	  return 0;
	}

a) 0	
b) 1000	
c) 999	
d) 998	
e) 1002	

4. Given the code snippet in the previous question, what is the maximum number of threads that can be there at any given moment?
a) 1
b) 2
c) 3
d) 4
e) None of the above

5. What value should <FREE> be in init(), for this code to work as a mutual exclusion primitive	typedef struct __lock_t {
	int flag;
	} lock_t;
	
	void init(lock_t *lock) {
	lock->flag = <FREE>;
	}
	void acquire(lock_t *lock) {
	while(xchg(&lock->flag, <HELD>) == <HELD>);
	// spin-wait (do nothing)
	}
	void release(lock_t *lock) {
	lock->flag = 1;
	}

a) 0	
b) 1	
c) 2	
d) 0 or 2	
e) It depends on <HELD> value	

CONTINUE THE QUESTIONS NEXT PAGE

Question 5) **(15 Points)**

a) Explain the purpose of the dummy node in Michael and Scott's concurrent queue implementation. Write the function that contains the dummy node and mark the line of this node with a comment.

b) Identify the error in the following modified version of List_Insert and write a corrected version

```
int List_Insert(list_t *L, int key) {
  pthread_mutex_lock(&L->lock);
  node_t *new = malloc(sizeof(node_t));
  if (new == NULL) {
    perror("malloc");
    return -1;
  }
  new->key = key;
  new->next = L->head;
  L->head = new;
  pthread_mutex_unlock(&L->lock);
  return 0;
}
```

c) For an approximate counter with threshold S=1024 running on 4 CPUs, trace through the number of global lock acquisitions required if each CPU increments its local counter 3000 times. Compare this to the number of lock acquisitions required for a precise counter.

It always seems impossible until it is done

Dr. Ahmed Hosny - 2025